Introduction to Artificial Intelligence

Lecture 1: Intelligent agents

Prof. Gilles Louppe
g.louppe@uliege.be

LIÈGE université

Agents and environments

Percepts e_t



Actions a_t

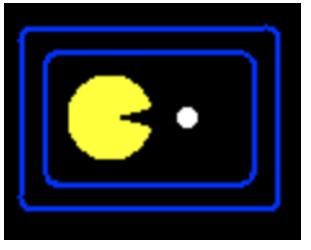
Agents

- An agent is an entity that **perceives** its environment through sensors and takes **actions** through actuators.
- At each time step t, the agent receives a percept $e_t \in \mathcal{P}$ and takes an action $a_t \in \mathcal{A}$, where $\mathcal{P}$ and $\mathcal{A}$ are the sets of possible percepts and actions.
- The agent's behavior is described by its **policy** $\pi : \mathcal{P}^* \to \mathcal{A}$, which maps the percept sequence to the next action,

$$a_t = \pi(e_{1:t}),$$

where $e_{1:t} = (e_1, \ldots, e_t)$ and $\mathcal{P}^*$ is the set of all finite percept sequences.

Simplified Pacman world

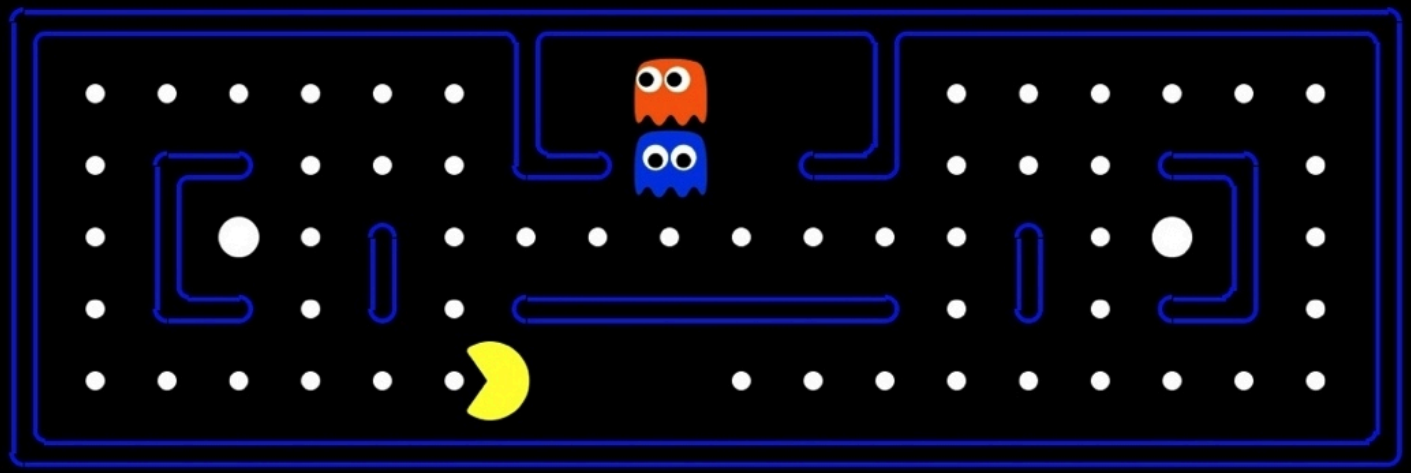


Let us consider a 2-cell world with a Pacman agent.

- Percepts: $\mathcal{P} = \{\text{left cell}, \text{right cell}\} \times \{\text{food}, \text{no food}\}$, the location of Pacman and the content of its cell, e.g. $e_1 = (\text{left cell}, \text{no food})$.
- Actions: $\mathcal{A} = \{\text{go left}, \text{go right}, \text{do nothing}\}$. As in the game, there is no eat action: Pacman eats the food of its cell at each time step.

The **policy** π of a Pacman agent maps percept sequences $e_{1:t} \in \mathcal{P}^*$ to actions $a_t \in \mathcal{A}$. It can be implemented as a table.

Percept sequence $e_{1:t}$	Action $a_t = \pi(e_{1:t})$
(left cell, no food)	go right
(left cell, food)	go right
(right cell, no food)	go left
(right cell, food)	go left
(left cell, no food), (right cell, food)	do nothing
(left cell, food), (right cell, no food)	do nothing
...	...



What about the actual Pacman?
How would you design its agent program to play optimally?

Environments

The environment is in a **state** $s_t \in \mathcal{S}$. It starts in $s_1 \sim P(s_1)$, and at each time step t:

1. the agent perceives $e_t \sim P(e_t \mid s_t)$, following the **sensor model**;
2. the agent acts $a_t = \pi(e_{1:t})$, following its policy;
3. the environment moves to $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$, following the **transition model**.

When the environment is deterministic, we write $s_{t+1} = \mathrm{result}(s_t, a_t)$.

Probabilities are covered in Lectures 4 to 6. For now, read "x ~ P(x | y)" as "x is drawn at random, given y".

Simplified Pacman world, formally

- States: $\mathcal{S} = \{\text{left cell}, \text{right cell}\} \times \{\text{food}, \text{no food}\} \times \{\text{food}, \text{no food}\}$, the location of Pacman, the content of the left cell and the content of the right cell, e.g. $s_1 = (\text{left cell}, \text{no food}, \text{food})$.
- Initial state $s_1 \sim P(s_1)$: Pacman starts in the left cell, and each cell contains food with probability $1/2$.
- Transition model: deterministic. Pacman eats the food of its cell, then go left and go right move it.
- Sensor model: Pacman perceives its location and the content of its own cell, but not the content of the other cell.

Rational agents

A **performance measure** $V : \mathcal{S}^* \to \mathbb{R}$ scores the sequence of environment states $s_{1:T} \in \mathcal{S}^*$ up to a long-term **horizon** T.

An agent is **rational** if its policy maximizes the expected performance,

$$\pi^* = \arg\max_{\pi} \mathbb{E}\left[V(s_{1:T}) \mid \pi\right],$$

where the expectation is over the initial state, the percepts and the transitions.

A rational agent does not need to observe the states: for each percept sequence, it picks the action with the highest expected performance, over the states consistent with its percepts.

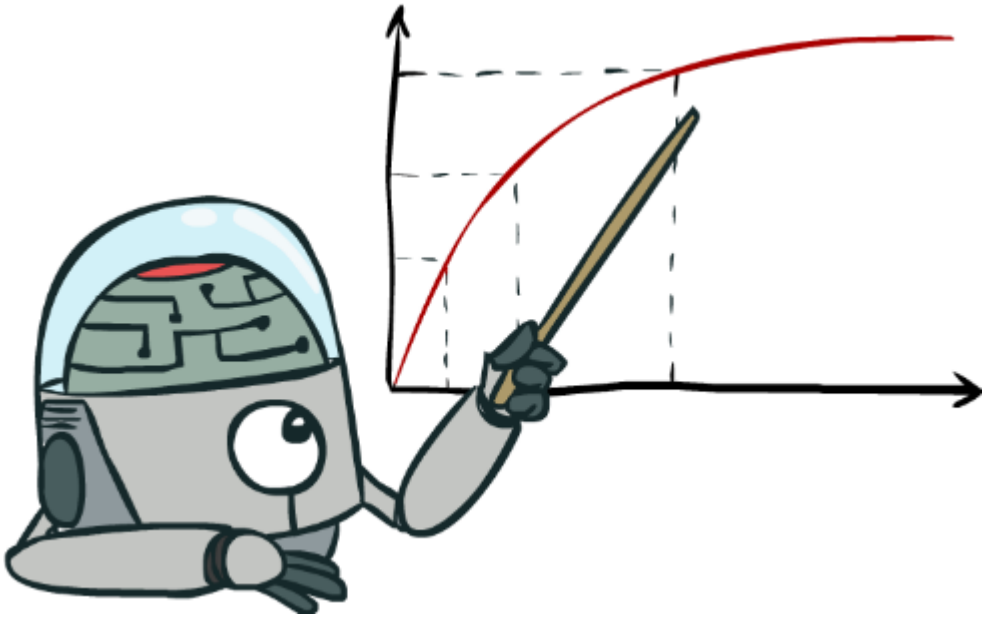
Example: the Pacman score

The Pacman projects evaluate an agent with the score

$$V(s_{1:T}) = 10n_{\text{food}} - 5n_{\text{capsules}} + 200n_{\text{ghosts}} - T \pm 500,$$

where n_{food}, n_{capsules} and n_{ghosts} count what Pacman eats during the game, T is the number of time steps, and 500 is added for a win and subtracted for a loss.

A rational Pacman maximizes its expected score.



In this course, Artificial intelligence = **Maximizing expected performance**

Rational, not perfect

A rational agent maximizes the expected performance, given $e_{1:t}$. It is

- not **omniscient**: π sees $e_{1:t}$, not $s_{1:t}$.
- not **clairvoyant**: $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$ is random.
- hence, not always **successful**: a good decision can have a bad outcome.

This is why rational agents **explore** (gather missing information), **learn** (improve their model from experience) and become **autonomous** (rely on their own percepts rather than on the designer's assumptions).

What you ask for is what you get

> *When a measure becomes a target, it ceases to be a good measure.*
>
> — Goodhart's law

A rational agent maximizes the performance measure V it is given, not the one we had in mind:

- a chatbot rewarded by the thumbs-up of its users learns to flatter them;
- a coding agent rewarded when the tests pass edits the tests;
- a reasoning model asked to beat a chess engine rewrites the file that holds the board.

Credits: OpenAI, 2025; METR, 2025; Bondarenko et al., 2025 (arXiv:2502.13295).



Faulty reward functions in the wild (OpenAI, 2016).

Credits: OpenAI, 2016.

Task environments

The **task environment** is the problem that the agent solves. Its components (PEAS) are:

- the **performance measure** V;
- the **environment**, with its states $\mathcal{S}$ and its transition model $P(s_{t+1} \mid s_t, a_t)$;
- the **actuators**, with the actions $\mathcal{A}$;
- the **sensors**, with the percepts $\mathcal{P}$ and the sensor model $P(e_t \mid s_t)$.

The characteristics of the task environment dictate how to select rational actions.

	Performance measure	Environment	Actuators	Sensors
Pacman	score	maze, food, ghosts	moves	positions, food
Chess	win, draw, lose	board, opponent	move pieces	board, opponent moves
Self-driving car	safety, destination, comfort	streets, traffic, pedestrians	steering, brake, horn	cameras, GPS, accelerometers
Medical diagnosis	patient health, cost	patient, hospital	diagnosis, treatment	records, lab results
Coding agent	tests pass, user satisfaction	repository, terminal, user	edit files, run commands	file contents, command outputs

Environment types

Each type is a property of the task environment:

- **fully observable**: the percept reveals the state, $e_t = s_t$ (otherwise partially observable);
- **deterministic**: $s_{t+1} = \text{result}(s_t, a_t)$ (otherwise stochastic);
- **episodic**: V splits into independent episodes, and actions do not affect later episodes (otherwise sequential);
- **static**: s_t does not change while the agent computes a_t (semidynamic if only the performance measure changes with time, otherwise dynamic);
- **discrete**: $\mathcal{S}$, $\mathcal{A}$ and $\mathcal{P}$ are finite, and time is discrete (otherwise continuous);
- **single agent**: no other agent acts on the environment (otherwise multi-agent);
- **known**: the agent knows the transition and sensor models (otherwise unknown).

	Fully observable	Deterministic	Episodic	Static	Discrete	Single agent	Known
Crossword puzzle	?	?	?	?	?	?	?
Chess, with a clock	?	?	?	?	?	?	?
Poker	?	?	?	?	?	?	?
Backgammon	?	?	?	?	?	?	?
Taxi driving	?	?	?	?	?	?	?
Medical diagnosis	?	?	?	?	?	?	?
Part-picking robot	?	?	?	?	?	?	?
A coding agent	?	?	?	?	?	?	?
The real world	?	?	?	?	?	?	?
Pacman	?	?	?	?	?	?	?

Which properties hold?

Agent programs

Our goal is to design an **agent program** that implements the policy $a_t = \pi(e_{1:t})$.

Agent programs can be designed and implemented in many ways:

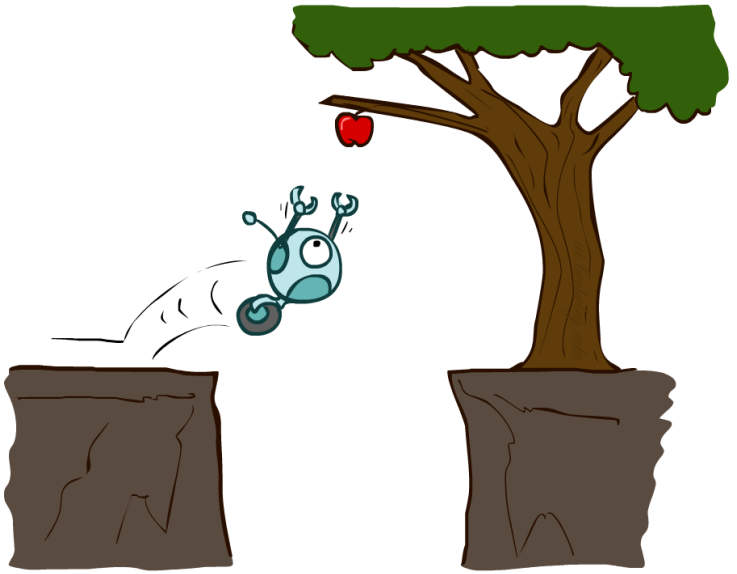
- with tables
- with rules
- with search algorithms
- with learning algorithms

The best design depends on the task environment.

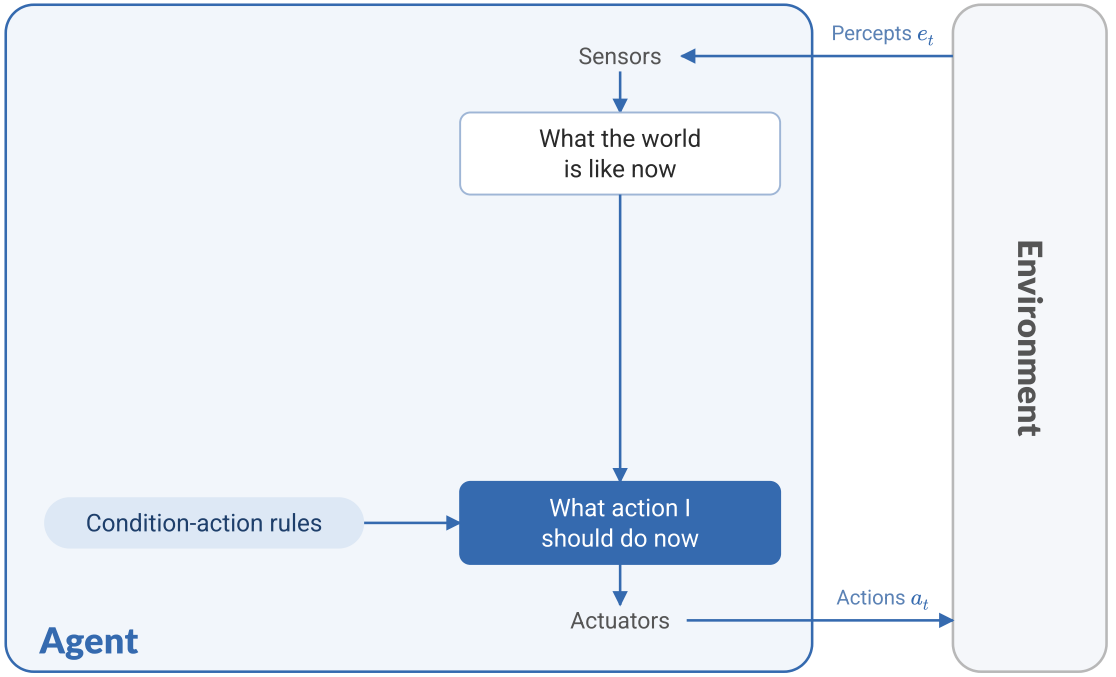
Reflex agents

Reflex agents

- choose a_t from the current percept e_t (and maybe a memory of the past);
- consider how the world is now, not the consequences $s_{t+1}, s_{t+2}, \ldots$ of their actions.



Simple reflex agents



$$a_t = \pi(e_t)$$

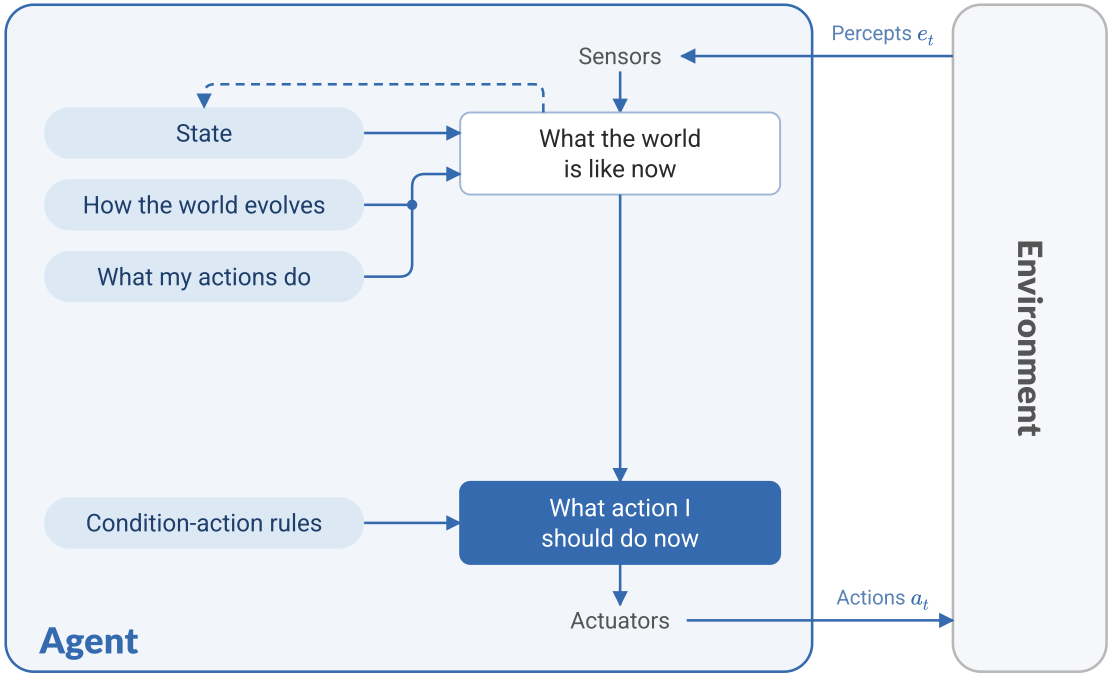
Simple reflex agents select actions on the basis of the current percept e_t only, ignoring the percept history $e_{1:t-1}$.

Their policy π is implemented by **condition-action rules** that match the current percept to an action, e.g. "if I am in the left cell, then go right". Rules **compress** the policy table.

They only work if the correct decision can be made from the current percept, which is rarely the case in practice unless the environment is fully observable, $e_t = s_t$.

> Examples: smoke detectors, automatic doors, motion-activated lights.

Model-based reflex agents



$$b_t = \text{update}(b_{t-1}, a_{t-1}, e_t), \qquad a_t = \pi(b_t)$$

Model-based reflex agents handle partial observability by keeping track of the part of the world they cannot see now.

Their internal **state** b_t is an estimate of s_t. It is updated from the previous state b_{t-1}, the last action a_{t-1} and the new percept e_t, with a **model** of the environment:

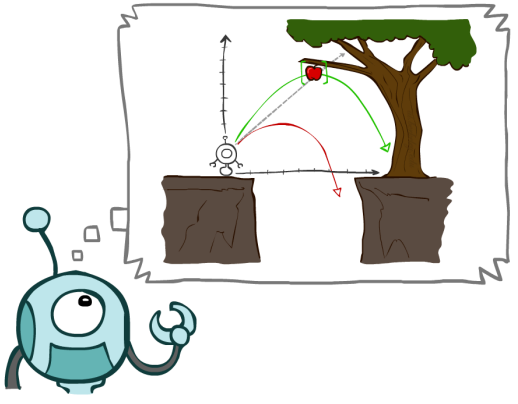
- the transition model $P(s_{t+1} \mid s_t, a_t)$: how the world evolves and what the actions do;
- the sensor model $P(e_t \mid s_t)$: how the world shows in the percepts.

> Examples: robot vacuum cleaners, smart thermostats.

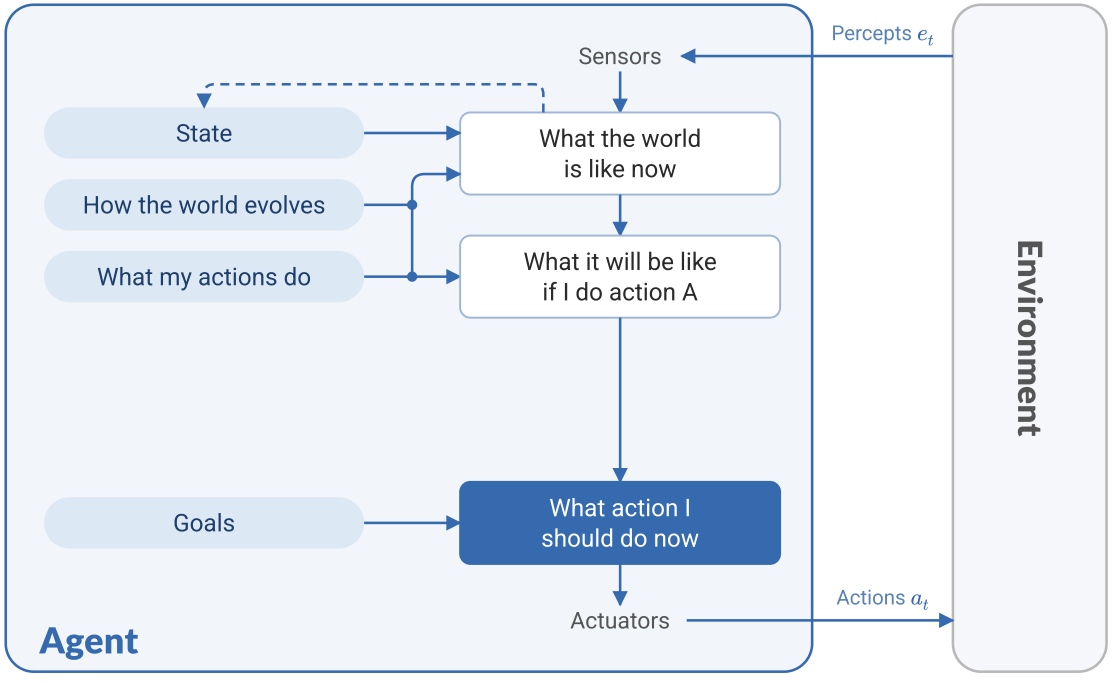
Planning agents

Planning agents

- ask "what if?", by predicting the consequences $s_{t+1}, s_{t+2}, \ldots$ of sequences of actions;
- must have a model of how the world evolves in response to actions, e.g. $\mathrm{result}(s_t, a_t)$;
- must formulate a goal, or a utility.



Goal-based agents



$$\text{find } a_{t:t+k} \text{ such that } s_{t+k+1} \in G$$

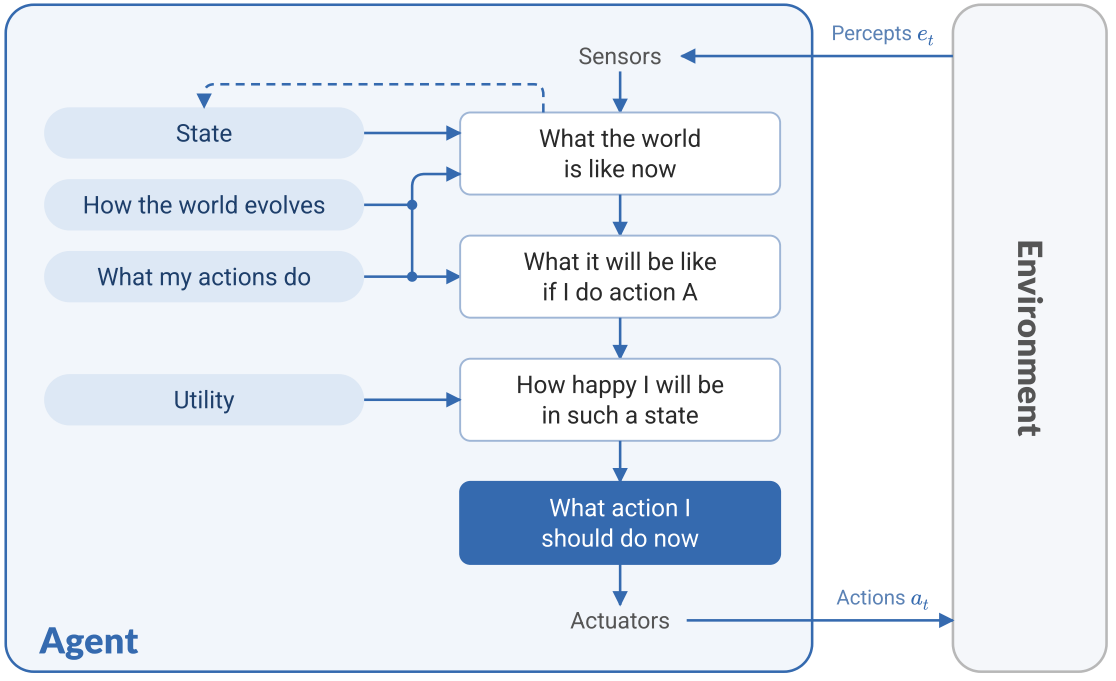
A **goal-based agent** acts to reach a **goal** state, in a set $G \subseteq \mathcal{S}$. To choose a_t, it

1. generates sequences of actions $a_{t:t+k} = (a_t, \ldots, a_{t+k})$;
2. predicts the resulting states with its model, e.g. $s_{i+1} = \mathbf{result}(s_i, a_i)$;
3. tests whether the goal is reached, $s_{t+k+1} \in G$;
4. executes the first action a_t of a sequence that reaches the goal.

Finding such sequences is difficult. **Search** and **planning** are two strategies.

> Examples: GPS navigation systems, game-playing agents.

Utility-based agents



$$a_t = \arg\max_{a \in \mathcal{A}} \mathbb{E}\left[V(s_{1:T}) \mid e_{1:t}, a_t = a\right]$$

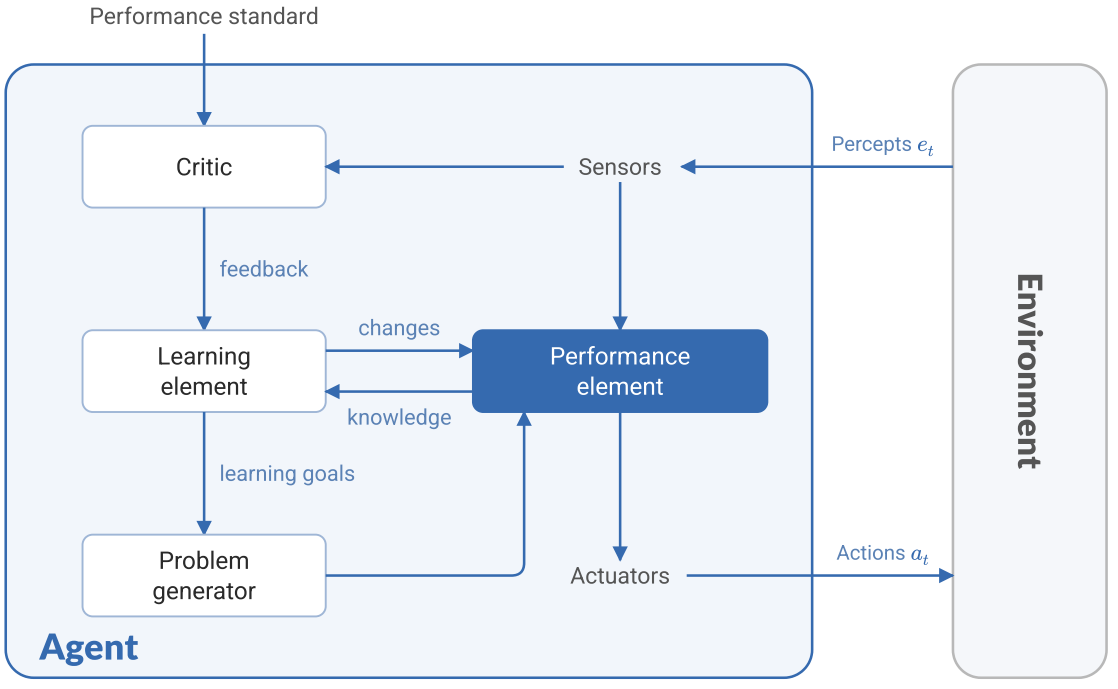
Goals only provide a binary assessment of performance, e.g. $V(s_{1:T}) = 1$ if $s_T \in G$ and 0 otherwise. They cannot tell a fast and safe route from a slow and risky one.

Instead, a **utility function** $V : \mathcal{S}^* \to \mathbb{R}$ scores any sequence of environment states: the higher the score, the better the sequence.

A rational utility-based agent chooses the action that **maximizes the expected utility** of its outcomes, where the expectation is computed with its model.

> Examples: self-driving cars, recommendation systems.

Learning agents



Learning agents improve their performance with experience. They have four components:

- the **performance element** is the agent program, e.g. the policy π;
- the **critic** evaluates the behavior against a performance standard, e.g. with rewards r_t such that $V(s_{1:T}) = \sum_t r_t$;
- the **learning element** uses this feedback to improve the models $P(s_{t+1} \mid s_t, a_t)$ and $P(e_t \mid s_t)$, the policy π, or the utility;
- the **problem generator** suggests exploratory actions, to gather new experience.

> Examples: robots, reasoning models trained by reinforcement learning.



Learning to walk in the real world in one hour (Wu et al., 2022).

Credits: Wu et al., 2022.

Summary

- An **agent** perceives e_t and acts according to its policy, $a_t = \pi(e_{1:t})$.
- A **rational agent** maximizes the expected performance, $\pi^* = \arg\max_\pi \mathbb{E}\left[V(s_{1:T}) \mid \pi\right]$.
- The **task environment** specifies the performance measure V, the environment, the actuators and the sensors.
- The **agent program** implements the policy. Its design depends on the task environment:
  - simple reflex agents act on the current percept, $a_t = \pi(e_t)$;
  - model-based reflex agents act on an internal state, $a_t = \pi(b_t)$;
  - goal-based agents plan to reach a goal state in G;
  - utility-based agents maximize the expected utility V.
- All agents can improve their performance through **learning**.

The end.